

Week 9 — External Libraries & APIs

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- What is an API and how do they work?
- HTTP status codes
- `requests.get()` and `.json()`
- Navigating nested JSON
- Handling request errors

Apply — ~30 min

- Code-along: first API request
- Code-along: loop through a JSON list
- Code-along: Country Explorer CLI starter

Everything Comes Together Here

Until now, all your data was typed into the code or read from a local file.

This week your programs **reach out to the internet** and pull in live data.

The tools are new — but the data structures aren't. Once you parse an API response, you're right back in familiar territory: dicts, lists, loops, and functions.

What Is an API?

An **API** (Application Programming Interface) is a way for programs to request data from a server over the internet.

- You send a **request** to a URL (an endpoint)
- The server sends back a **response** — usually JSON
- Your code parses the JSON and works with the data

HTTP Status Codes

Every response comes with a status code.

Code	Meaning
200	OK — request succeeded
400	Bad Request — something wrong with your request
404	Not Found — endpoint or resource doesn't exist
429	Too Many Requests — you're being rate-limited
500	Server Error — the API's problem, not yours

Making a GET Request

`requests.get()` fetches data from a URL.

```
import requests

response = requests.get("https://api.agify.io/?name=michael")
print(response.status_code)      # 200
print(response.json())           # {'name': 'michael', 'age': 40}
```

Check for Understanding #1

What does `response.json()` return?

- A) A string of raw JSON text
- B) A Python dict or list, depending on the API response
- C) An HTTP status code
- D) A file object

Navigating Nested JSON

Real API responses are often nested dicts inside lists.

```
response = requests.get(
    "https://restcountries.com/v3.1/region/europe"
    "?fields=name,population"
)
countries = response.json()    # list of dicts

for country in countries[:5]:
    name = country["name"]["common"]
    pop  = country["population"]
    print(f"{name}: {pop:,}")
```

Query Parameters

Pass parameters to filter or customize the response.

```
params = {"name": "michael", "country_id": "US"}  
response = requests.get("https://api.agify.io/", params=params)
```

Check for Understanding #2

The REST Countries API returns this for one country:

```
{"name": {"common": "Finland", "official": "Republic of Finland"},  
  "capital": ["Helsinki"], "population": 5530719}
```

How do you access the common name?

- A) `country["name"]`
- B) `country["name"]["common"]`
- C) `country["common"]`
- D) `country[0]["name"]`

Handling Request Errors

Wrap API calls in try/except and check the status code.

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
except requests.exceptions.RequestException as e:
    print(f"Request failed: {e}")
    data = []
```

Check for Understanding #3

You make an API call and `response.status_code` is `404`. What should your program do?

- A) Parse `response.json()` as normal — 404 responses always have data
- B) Print an error message and skip parsing — 404 means the resource wasn't found
- C) Retry the request immediately
- D) Ignore the status code — it doesn't matter

Time to Apply

Let's fetch and work with live API data.

Mentor 2 takes over

Assignment 9 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — first GET request, print status + full JSON
- `warmup2.py` — access specific fields with `.get()` + fallback
- `warmup3.py` — loop through a JSON list, print first 10 results
- `warmup4.py` — wrap a request in try/except + check status code

Part 2 — Country Explorer CLI

- Fetch all countries from REST Countries API on startup
- Menu: search by name (partial match), filter by region (sorted by pop)
- Handle missing capital fields with `.get()` or conditional
- Wrap the initial request in try/except

Code-Along 1: First API Request

Fetch from a public API and inspect the response.

```
import requests

url = "https://api.agify.io/?name=michael"
response = requests.get(url)

print("Status:", response.status_code)
print("Data: ", response.json())

data = response.json()
print(f"Predicted age for {data['name']}: {data['age']}")
```

Code-Along 2: Loop Through a JSON List

Fetch countries and print just the names.

```
import requests

url = "https://restcountries.com/v3.1/region/europe"
response = requests.get(url, params={"fields": "name,population"})
countries = response.json()

for country in countries[:10]:
    print(country["name"]["common"])
```

Code-Along 3: Country Explorer — Fetch + Parse

Build the data loading function the whole CLI will use.

```
import requests

def fetch_countries():
    url = "https://restcountries.com/v3.1/all"
    params = {"fields": "name,capital,region,population"}
    try:
        r = requests.get(url, params=params, timeout=10)
        r.raise_for_status()
        return [{"name": c["name"]["common"],
                  "capital": c.get("capital", ["N/A"])[0],
                  "region": c["region"],
                  "population": c["population"]}
                for c in r.json()]
    except requests.exceptions.RequestException as e:
        print(f"Failed to load data: {e}")
        return []
```

Extension Challenge

Ungraded extension: Add a third menu option — "Top 10 most populous countries" — that displays the 10 countries with the largest population, sorted largest first, regardless of region.

Can you do it in two lines inside the menu handler?

Wrap-Up

Today you learned:

- What APIs are and how HTTP requests work
- `requests.get()` and `response.json()`
- Navigating nested JSON — the same dicts and lists you already know
- Handling request errors with `try/except` and `raise_for_status()`

This week: Complete warmups 1–4 and the Country Explorer. Submit via pull request.

Next week: Final Project Part I — you'll build your own API-powered CLI program.